

to instantiate your type parameters. However, all types are subclasses of `Object`, so we can call methods of `Objects` on values of generic type. Note, in Java we must use wildcard type parameters to get the same effect.

```
public partial struct Pair<T, U>
{
    public override string ToString()
    {
        return "(" + Fst.ToString() + ", " + Snd.
ToString() + ")";
    }
}
```

If we want to expand on the methods we can call on objects of generic type, we must refine our requirements on how we can instantiate our type parameters by constraints. For example, if we want to implement a sort function over a generic list, we'll need to require that the list contains elements that implement `IComparable`.

```
public static class BubbleSort
{
    public static void Sort<T>(IList<T> l) where T :
IComparable<T>
    {
        for (int i = 0; i < l.Count; i++)
        {
            for (int j = i+1; j < l.Count; j++)
            {
                if (l[j].CompareTo(l[i]) < 0)
                {
                    T temp = l[i];
                    l[i] = l[j];
                    l[j] = temp;
                }
            }
        }
    }
}
```

We can constraint type parameters to implement a particular interface or extend a particular class. In addition to this, we can also restrict a type parameter to be instantiated only with a class or struct type. For example,